ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

STOCKFLOW

Logiciel de gestion de stock

pour un magasin de vente d'électronique

PRÉPARÉ PAR :

- Abouelfadile Nassim
- Mellati Mohamed Amine
- Louriki Reda

ENCADRÉ PAR :

- Pr. Mustapha Hain

FILIÈRE : IAGI-1
ANNÉE ACADÉMIQUE : 2026 – 2027

REMERCIEMENTS

Remerciements

Nous tenons à remercier très sincèrement notre professeur, **Pr. Mustapha Hain**, pour son encadrement tout au long de ce projet.

Grâce à son dossier de conception clair et précis, nous savions exactement ce que chaque écran devait faire, et nous avons pu nous concentrer sur l'essentiel : comprendre notre code et le construire nous-mêmes, étape par étape. Nous le remercions aussi de nous avoir fait travailler avec GitHub : cette façon de travailler nous a appris à collaborer comme une vraie équipe de développement, où le travail de chacun est visible.

Merci pour sa disponibilité, ses conseils et la confiance qu'il nous a accordée. Ce projet nous a beaucoup apporté, et nous en sommes fiers.

Nous remercions également l'**École Nationale Supérieure d'Arts et Métiers de Casablanca** pour le cadre de formation qu'elle nous offre.

Abouelfadile Nassim
Mellati Mohamed Amine
Louriki Reda

SOMMAIRE

Remerciements 2

Introduction 4

Interface 1 : Articles / Stock 5

Interface 2 : Clients et Fournisseurs 6

Interface 3 : Réceptions 7

Interface 4 : Livraisons 8

Conclusion 9

Introduction

Stockflow est un logiciel de gestion de stock pour un magasin de vente d'électronique : téléviseurs, réfrigérateurs, lave-linge, ordinateurs et accessoires. Il suit le parcours de la marchandise dans le magasin :

1. le magasin **achète** ses articles à des **fournisseurs** : chaque arrivée de marchandise est saisie dans un bon de **réception**, qui fait entrer les quantités en stock et met à jour le prix d'achat ;
2. il **vend** ces articles à ses **clients** : chaque vente est saisie dans un bon de **livraison**, qui fait sortir les quantités du stock, et qui est refusé si le stock ne suffit pas ;
3. à tout moment, l'écran **Articles / Stock** montre pour chaque article ce qui est entré, ce qui est sorti et le stock réel, met en rouge les articles en rupture, et calcule la valeur du stock.

L'application est écrite en Python avec `tkinter` et enregistre ses données dans une base **SQLite** (`stockflow.db`), en suivant le dossier de conception fourni.

Répartition du travail

Chaque fenêtre a été construite en deux temps : d'abord l'interface (le *front end*), testée avec des données fictives, puis la connexion à la base de données et les fonctions des boutons.

Fenêtre	Interface	Base de données
1. Articles / Stock	Abouelfadile Nassim	Louriki Reda
2. Clients et Fournisseurs	Mellati Mohamed Amine	Louriki Reda
3. Réceptions	Abouelfadile Nassim	Louriki Reda
4. Livraisons	Mellati Mohamed Amine	Louriki Reda

Nous avons travaillé sur GitHub (github.com/nassim-0x/stockflow), chacun depuis son compte, avec un commit par fenêtre.

La base de données

La base SQLite (`stockflow.db`) contient seulement **trois tables**, créées dans `database.py` :

- `FOURNISSEUR` : code, nom, adresse et téléphone des fournisseurs ;
- `CLIENT` : code, nom, adresse et téléphone des clients ;
- `STOCK` : les articles, avec leurs prix, les quantités entrées et sorties, et le stock réel.

Les réceptions et les livraisons ne créent pas de nouvelle table : elles mettent à jour les quantités de la table `STOCK`.

Comment lire ce rapport

Chaque fenêtre est présentée sur une page : une capture de l'application réelle avec des **cadres rouges numérotés**, l'explication de chaque cadre, puis l'essentiel du code. Une étiquette bleue indique qui a écrit chaque partie.

INTERFACE 1 : ARTICLES / STOCK

C'est l'écran principal du magasin : la liste de tous les articles avec leurs prix et leur stock, et une fiche pour ajouter ou modifier un article.



Ce que montre l'écran

1. La famille de l'article (Téléviseur, Informatique, Électroménager) se choisit dans une liste.
2. Les boutons pour créer, enregistrer et supprimer un article.
3. Un article en rupture de stock s'affiche en rouge.
4. La recherche filtre la liste à chaque lettre tapée.
5. La valeur totale du stock (prix d'achat × stock réel).

L'essentiel du code

Abouelfadile Nassim *Interface*

4 `articles.py` – lignes 112 à 122

```
112      for item_id in tous_les_articles:
113          values = articles_table.item(item_id, "values")
114
115          reference = values[0].lower()
116          designation = values[1].lower()
117
118          # if text match we show the row,else hide it
119          if recherche in reference or recherche in designation:
120              articles_table.reattach(item_id, "", "end")
121          else:
122              articles_table.detach(item_id)
```

Pour chaque article, si le texte cherché se trouve dans la référence ou la désignation, la ligne reste visible ; sinon elle est cachée.

Louriki Reda *Base de données*

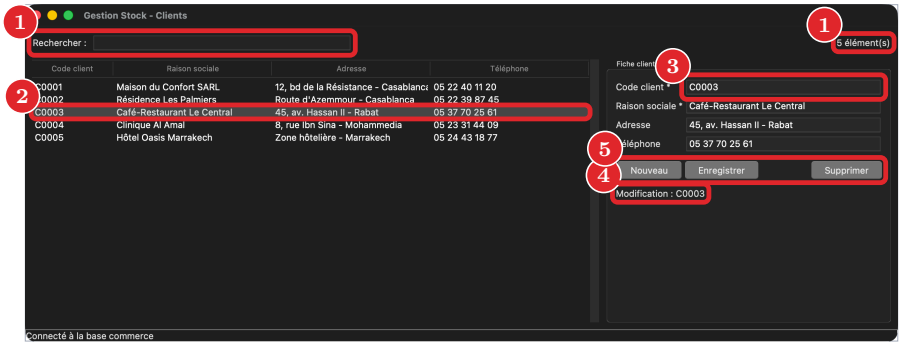
3 `articles.py` – version finale, lignes 70 à 74

```
70      for article in articles_db.lister():
71          couleur = ()
72          if article["STOCK_REEL"] <= 0:
73              couleur = ("rupture",)
74          articles_table.insert("", "end", iid=article["REF"], tags=couleur, values=(
```

3 Les articles sont lus dans la table `STOCK` de la base. Si le stock réel est à 0 ou moins, la ligne reçoit la couleur rouge.

INTERFACE 2 : CLIENTS ET FOURNISSEURS

Les écrans Clients et Fournisseurs ont la même présentation : une liste avec une recherche et une fiche de saisie. On présente ici l'écran des clients.



Ce que montre l'écran

1. La recherche, et le nombre de clients trouvés.
2. Un clic sur un client remplit la fiche avec ses informations.
3. Le code client est calculé automatiquement (`C0006` après `C0005`).
4. La fiche indique si on crée un client ou si on le modifie.
5. Les boutons Nouveau, Enregistrer et Supprimer.

L'essentiel du code

Mellati Mohamed Amine *Interface*

3 `clients.py` – lignes 34 à 41

```
34  def prochain_code():
35      """Cherche le plus grand code et ajoute 1 : C0005 -> C0006."""
36      plus_grand = 0
37      for code in clients:
38          numero = int(code[1:])
39          if numero > plus_grand:
40              plus_grand = numero
41      return "C" + str(plus_grand + 1).zfill(4)
```

On cherche le plus grand numéro de client et on ajoute 1, écrit sur 4 chiffres.

Louriki Reda *Base de données*

`clients_db.py` – version finale, lignes 12 à 17

```
12  def enregistrer(code, nom, adresse, tel):
13      # le code existe déjà -> modification, sinon -> nouveau client
14      if len(lire("select * from CLIENT where CODE_CLIENT = ?", (code,))) > 0:
15          return executer("update CLIENT set CNAME = ?, ADDRESSE = ?, TELEPHONE =
         ? where CODE_CLIENT = ?",
16                          (nom, adresse, tel, code))
17      return executer("insert into CLIENT values (?, ?, ?, ?)", (code, nom,
         adresse, tel))
```

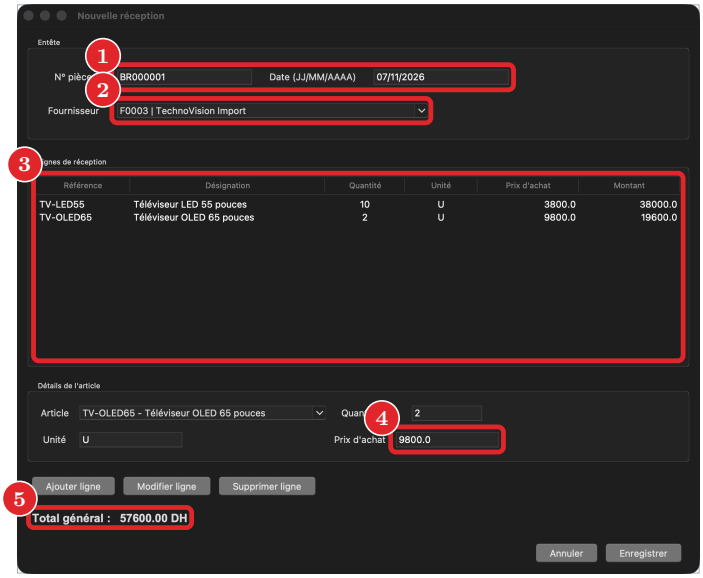
Si le code existe déjà dans la table `CLIENT`, le client est modifié ; sinon, il est ajouté.



Un fournisseur qui a déjà livré des articles ne peut pas être supprimé.

INTERFACE 3 : RÉCEPTIONS

Une réception, c'est la marchandise achetée à un fournisseur qui arrive au magasin. Le bouton **Nouvelle réception** ouvre cette fenêtre de saisie.



Ce que montre l'écran

1. Le numéro du bon est calculé automatiquement, avec la date du jour.
2. Le fournisseur se choisit dans une liste.
3. Les articles reçus, avec leur quantité et leur prix.
4. Le prix d'achat de l'article est proposé automatiquement.
5. Le total du bon est recalculé à chaque ligne ajoutée.

L'essentiel du code

Abouelfadile Nassim *Interface*

`reception.py` – lignes 26 à 29

```
26  def ouvrir_popup():
27      popup = tk.Toplevel(window)
28      popup.title("Nouvelle réception")
29      popup.geometry("950x650")
```

La saisie s'ouvre dans une deuxième fenêtre (`Toplevel`), au-dessus de la liste des réceptions.

Louriki Reda *Base de données*

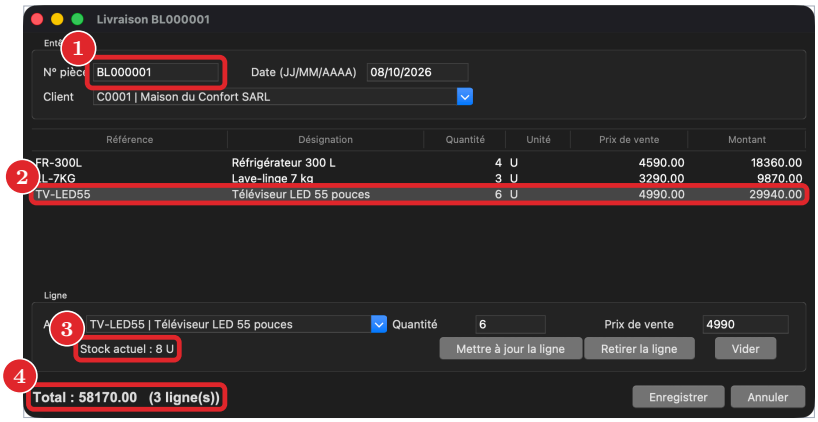
`mouvements_db.py` – version finale, lignes 7 à 12

```
7   def receptionner(lignes, fournisseur):
8       """Réception : les quantités entrent en stock, le prix d'achat et le fournisseur sont mis à jour."""
9       for ref in lignes:
10          quantite, prix = lignes[ref]
11          executer("update STOCK set ENTRE = ENTRE + ?, STOCK_REEL = STOCK_REEL + ?, PRIX_ACHAT = ?, FCODE = ? where REF =
        ?",
12                   (quantite, quantite, prix, fournisseur, ref))
```

À l'enregistrement, les quantités reçues **entrent** dans le stock, et le prix d'achat de l'article est mis à jour.

INTERFACE 4 : LIVRAISONS

Une livraison, c'est la marchandise vendue à un client qui sort du magasin. L'écran fonctionne comme les réceptions, mais une livraison est refusée si le stock ne suffit pas.



Ce que montre l'écran

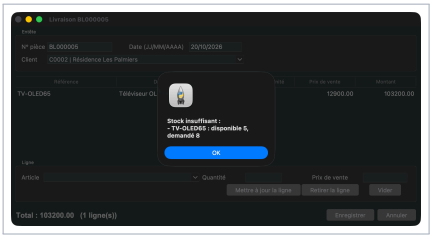
1. Le numéro du bon de livraison est calculé automatiquement.
2. Un clic sur une ligne permet de la modifier.
3. Le stock actuel de l'article choisi s'affiche.
4. Le total du bon est recalculé à chaque changement.

L'essentiel du code

Mellati Mohamed Amine *Interface*

`livraisons.py` – lignes 212 à 221

```
212     problemes = ""
213     for ref in lignes_en_cours:
214         quantite = lignes_en_cours[ref][0]
215         stock = articles[ref][2]
216         if quantite > stock:
217             problemes = problemes + "\n - " + ref + " : disponible " + str(stock) + ", demandé " + str(quantite)
218
219     if problemes != "":
220         messagebox.showerror("Livraison refusée", "Stock insuffisant :" + problemes, parent=saisie)
221         return
```



Avant d'enregistrer, chaque ligne est comparée au stock. S'il en manque, un seul message liste tous les articles en défaut, et rien n'est enregistré.

Louriki Reda *Base de données*

`mouvements_db.py` – version finale, lignes 22 à 27

```
22  def livrer(lignes):
23      """Livraison : les quantités sortent du stock."""
24      for ref in lignes:
25          quantite = lignes[ref][0]
26          executer("update STOCK set SORTIE = SORTIE + ?, STOCK_REEL = STOCK_REEL - ? where REF = ?",
27                   (quantite, quantite, ref))
```

À l'enregistrement, les quantités livrées **sortent** du stock.

Conclusion

Ce projet a été pour nous une expérience vraiment enrichissante. En partant d'un simple dossier de conception, nous avons construit ensemble **Stockflow**, un logiciel complet de gestion de stock pour un magasin d'électronique, qui fonctionne de bout en bout : on reçoit la marchandise des fournisseurs, on la livre aux clients, et le stock se met à jour tout seul.

Ce que nous avons appris

Au-delà du résultat, ce projet nous a surtout permis de **comprendre** ce que nous faisions :

- **Construire une interface graphique** avec `tkinter` : choisir entre `pack` et `grid`, utiliser les `Treeview`, les `Combobox` et les fenêtres `Toplevel`.
- **Rendre une interface vivante** : relier les boutons et les événements à des fonctions, et contrôler chaque saisie pour que l'application ne plante jamais.
- **Utiliser une base de données** : écrire des requêtes SQL sûres et séparer le SQL de l'interface dans des modules dédiés.
- **Travailler en équipe avec Git et GitHub** : se répartir les fenêtres, construire sur le travail des autres sans l'écraser, et garder un historique clair de qui a fait quoi.

Le plus marquant pour nous a été de voir les trois parties se rejoindre : une interface dessinée d'abord avec des données fictives est devenue, une fois reliée à la base, un vrai logiciel utilisable par un magasin. Chacun de nous peut aujourd'hui expliquer chaque ligne de sa partie, et comprend comment elle s'assemble avec celle des autres.

Nous sommes fiers du résultat, et ce projet nous a donné envie d'aller plus loin. Nous remercions encore une fois **Pr. Mustapha Hain** pour son accompagnement tout au long de ce travail.